CivicOne System Architecture & Algorithms Specification

This document provides a comprehensive technical breakdown of the **CivicOne Digital Identity & Citizen Portal System**. It covers the overall system architecture, data flow, security model, and detailed step-by-step algorithms for every login gateway and web page/view within the application.

---

1. System Architecture Overview

CivicOne is built as a full-stack, decoupled Web application featuring a modern React SPA frontend and a RESTful Express.js backend.

```
graph TD
    Client["React 18 SPA (Vite Client - Port 3000)"]
    API["Express.js REST API (Node Server - Port 3001)"]
    DB[("Mock Database Store (mockDb.js)")]

    Client -->|HTTP / JSON REST API| API
    API -->|CRUD & Audit Logging| DB
```

Key Architectural Layers

- **Frontend Layer (`src/`)**:

  - **Framework**: React 18 with Vite bundling.

  - **Styling**: Vanilla CSS visual design system featuring Glassmorphism, CSS Variables (`data-theme`), and responsive flex/grid layouts.

  - **Iconography**: Lucide React.

  - **Routing Strategy**: Single Page Application (SPA) view-state routing managed via `src/App.jsx` with hash and pathname synchronization (`#org` , `#authority` , `#admin` , `/verify?token=...`). Includes keyboard shortcuts (e.g., `Ctrl + Shift + A` for Super Admin access).

- **Backend API Layer (`server/index.js`)**:

- **Runtime**: Node.js v20+ with ES Modules (`type: "module"`).

- **Framework**: Express.js with CORS enabled for cross-origin requests.

- **Endpoints**:

- `/api/auth/*` : OTP generation, OTP verification, Aadhaar tokenized identity verification, Government Officer authentication, and Super Admin master login.

- `/api/card/*` : Dynamic citizen card retrieval, tier upgrades, dynamic QR token verification, and credential sharing.

- `/api/vault/*` : Vault document search, multi-category filtering, OCR auto-extraction upload, and cryptographic verification.

- `/api/services/*` : Departmental integration endpoints, entitlement sync, and MFA/consent management.

- `/api/admin/*` : System health metrics, administrative verification of Gold Pass applications, audit logs, and fraud detection.

- `/api/payment/*` : Payment gateway order creation and server-to-server webhook verification.

- **Data & Security Layer (`server/mockDb.js` & `src/data/mockData.js`)**:

- **State**: Centralized in-memory database store supporting multi-tenant demo citizen profiles, identity records, vault documents, notifications, and security audit logs.

- **Security Features**: Tokenized identity verification, masked Aadhaar numbers (`XXXX XXXX 8942`), biometric consent validation, and SHA-256 cryptographic QR code signatures.

---

2. Authentication & Login Algorithms

CivicOne provides 3 distinct login gateways plus 1 interactive demo session switcher algorithm.

2.1 Citizen Login Algorithm (`PreEntryGate.jsx`)

The Citizen Login uses a 5-stage zero-trust authentication flow:

```
sequenceDiagram
    autonumber
    actor Citizen
    participant Gate as PreEntryGate
    participant Server as Express Server
```

```
    participant DB as Mock DB

    Citizen->>Gate: 1. Enter 10-Digit Mobile Number
    Gate->>Server: POST /api/auth/send-otp { phone }
    Server-->>Gate: 200 OK { success: true, demoOtp: "123456" }
    Gate-->>Citizen: Render OTP Verification Step (60s Timer)

    Citizen->>Gate: 2. Input 6-Digit Mobile OTP ("123456")
    Gate->>Server: POST /api/auth/verify-otp { phone, otp }
    Server-->>Gate: 200 OK { sessionToken, requireIdentityVerification: true }
    Gate-->>Citizen: Render Aadhaar Identity Consent Screen

    Citizen->>Gate: 3. Grant Explicit Aadhaar Consent & Submit
    Gate-->>Citizen: Render Aadhaar Identity OTP Screen

    Citizen->>Gate: 4. Input 6-Digit Aadhaar OTP ("123456")
    Gate->>Server: POST /api/auth/identity-verify { consent: true, aadhaarOtp }
    Server->>DB: Record Security Audit Log & Return Verified Profile
    Server-->>Gate: 200 OK { citizen, identityStatus: "VERIFIED" }

    Gate->>Gate: 5. Execute 2.2s Device Security & Biometric Scan Animation
    Gate->>Citizen: Transition to CitizenDashboard View
```

Step-by-Step Algorithm:

- **Input Mobile Number**:

  - Validate that input contains exactly 10 numeric digits.

  - Dispatch `POST /api/auth/send-otp` with formatted phone number (`+91 XXXXXXXXXX`).

  - Initialize 60-second resend countdown timer and transition step to `OTP` .

- **Verify Mobile OTP**:

  - Validate that 6 auto-advancing digit fields are completed.

  - Dispatch `POST /api/auth/verify-otp` .

  - On success, set `sessionToken` and transition step to `IDENTITY_CONSENT` .

- **Identity Consent & Aadhaar Authorization**:

  - Require explicit checkbox selection for UIDAI tokenized verification.

  - On confirmation, transition step to `IDENTITY_OTP` .

- **Submit Identity OTP**:

  - Collect 6-digit Aadhaar authorization OTP.

- Dispatch `POST /api/auth/identity-verify`.

- Backend appends a security audit entry to `db.securityLogs` and returns verified citizen details.

- **Device Security Verification**:

- Transition to `DEVICE_VERIFY` state.

- Display a 2.2-second pulse animation performing device hash integrity, location check, and cryptographic handshake.

- Invoke `onAuthenticated(citizenData)` to load the `CitizenDashboard`.

2.2 Government Officer Login Algorithm (`AuthorityGate.jsx`)

The Officer Gate provides department-level authentication for government issuers:

Step-by-Step Algorithm:

- **Input Credentials**:

- Capture official email address (e.g., `officer.sharma@parivahan.gov.in`).

- Select issuing department from dropdown (e.g., *Parivahan Sewa*, *NHA*, *ITD*, *MEA*, *NAD*, *State Land Records*).

- Optional: Enter Badge ID and Officer Passcode (`govt123`).

- **Submission & Verification**:

- Dispatch `POST /api/auth/authority-login` with credentials.

- Backend verifies email domain and passcode.

- Generates an official session token (`GOVT-AUTH--SECURE`) with `LEVEL-3 VERIFIED` clearance.

- Appends login record to government security audit log.

- **Portal Entry**:

- Invoke `onAuthenticated(officerData)` and transition application view to `AuthorityPortal`.

2.3 Super Admin Login Algorithm (`AdminGate.jsx`)

The Super Admin Gate controls system-wide governance:

Step-by-Step Algorithm:

- **Shortcut / Route Trigger**:

  - Accessed via `#admin`, `/owner-admin`, or global keyboard shortcut (`Ctrl + Shift + A`).

- **Credential Capture**:

  - Capture Master Admin Username (e.g., `superadmin@civicone.gov.in`), Root Passkey (`superadmin123`), and Hardware Key Token (`MASTER-HW-KEY-9048`).

- **Root Authentication**:

  - Dispatch `POST /api/auth/admin-login`.

  - Backend verifies master clearance and assigns `MASTER ROOT CLEARANCE`.

  - Logs event into central security logs.

- **Portal Entry**:

  - Invoke `onAuthenticated(adminData)` and transition application view to `AdminPortal`.

2.4 Interactive Demo Account Switcher Algorithm (`CitizenDashboard.jsx`)

CivicOne allows instant switching between demo citizen accounts without logging out:

Step-by-Step Algorithm:

- User selects a demo citizen from the header dropdown (e.g., *Rajesh Kumar*, *Priya Sharma*, *Amit Patel*).
- Client sends `POST /api/citizen/switch-demo` with target `citizenId`.
- Backend updates `db.activeCitizenId` and fetches the new citizen's profile, virtual card data, and vault documents.
- Appends a new audit log: `"Switched Demo Session to [Name]"`.
- Frontend updates local states (`currentCitizen`, `cardData`, `documents`) reactively.

---

3. Web Page & View Algorithms

Below are the algorithms for all 10 core views and components in CivicOne.

3.1 Landing Page (`LandingPage.jsx`)

- **Purpose**: Public portal homepage highlighting CivicOne features, trust metrics, department integrations, and entrance portals.
- **Algorithm**:

  1. On render, display Hero Section with live counter stats (100M+ Citizens, 45+ Depts, 99.9% Uptime).

  2. Provide interactive portal selection cards:

  - **Citizen Portal**: Triggers `onAccessCivicOne()` -> Sets view to `gate`.

  - **Government Officer Portal**: Triggers `onOpenAuthorityPortal()` -> Sets view to `authority-gate`.

  - **Super Admin Portal**: Triggers `onOpenOwnerAdmin()` -> Sets view to `admin-gate`.

  3. Render feature showcase grids (Dynamic QR, Cryptographic Vault, Instant Verification).

3.2 Citizen Dashboard (`CitizenDashboard.jsx`)

- **Purpose**: Main authenticated hub for citizens with tab navigation, quick stats, notification drawer, and interactive account switcher.
- **Algorithm**:

  1. **Initialization (`useEffect`)**:

  - Execute `Promise.all()` fetching `/api/card/me`, `/api/vault/documents`, `/api/notifications`, `/api/updates/govt`, `/api/updates/news`, and `/api/citizens/demo`.

  - Populate React state hooks with returned data or fallback seed data.

  2. **Tab Routing**:

  - Handle active tab state (`home`, `card`, `vault`, `services`, `gold-pass`, `activity`, `govt-updates`, `news`, `security`, `help`, `profile`).

  - Render header with search bar, notification drawer toggle, theme switcher, and demo citizen selector.

  3. **Global Search**:

  - Filter documents, services, and news against query string in real-time.

3.3 Virtual Civic Card & Gold Pass Upgrade (`VirtualCard.jsx` & `GoldPassPaymentModal.jsx`)

- **Purpose**: Displays the citizen's dynamic digital identity card, dynamic QR verification code, and Gold Pass payment workflow.
- **Algorithm**:

  1. **Render Card**:

  - Display holder photo, masked Aadhaar, Civic ID, security chip ID, and tier badge (*Verified Citizen* vs. * 👑 Premium Gold Citizen*).

  2. **Dynamic QR Code Generation**:

  - Generate SHA-256 verification signature and verification URL (`http://localhost:3001/verify?token=...`).

  - Clicking "Verify QR" navigates to the public verification view.

  3. **Gold Pass Upgrade & Webhook Payment Algorithm**:

  ```mermaid

  graph TD

  A[Click Upgrade to Gold Pass] --> B[Open GoldPassPaymentModal]

  B --> C[Select Plan: ₹499/yr & Payment Method: UPI/Card]

  C --> D[POST /api/payment/create-order]

  D --> E[Simulate Payment Gateway Confirmation]

  E --> F[POST /api/payment/webhook]

  F --> G[Server Grants Gold Entitlement & Adds Audit Log]

  G --> H[Update Card Tier to GOLD reactively]

  ```

3.4 My Civic Vault (`CivicVault.jsx`)

- **Purpose**: Cryptographic document storage, filtering, upload, and instant verification against issuing authorities.
- **Algorithm**:

  1. **Filtering & Search**:

  - Send query params `category` , `status` , `search` , `sort` , and `favoritesOnly` to `GET /api/vault/documents` .

  - Render document cards with verification seals, issue dates, and ref numbers.

2. **Document Upload & Auto-Extraction**:

- Form collects Name, Category, Issuer, Ref No, and Privacy Toggle.

- On submission, send `POST /api/vault/upload`.

- Server generates security seal (`CIVIC-SEAL-VERIFIED-`) and appends document to vault.

3. **Instant Credential Verification**:

- User clicks "Verify Credential".

- Send `POST /api/vault/verify-doc/:id`.

- Server checks credential status against simulated issuer API, updates status to `Verified`, and returns verification confirmation.

3.5 Department Services & Schemes (`ServicesSection.jsx`)

- **Purpose**: Portal for accessing integrated government departments (Parivahan, NHA, Income Tax, Passport, Digilocker, Land Records).
- **Algorithm**:

1. Display department service cards categorized by ministry.

2. **Department Sync**:

- User clicks "Sync Services".

- Send `POST /api/services/category/:catKey/sync`.

- Server updates `lastSynced` timestamp and returns synced document count.

3. **Consent & MFA Management**:

- User toggles consent for automated data sharing between departments.

- Send `POST /api/services/category/:catKey/consent`.

3.6 Public Dynamic QR Verification (`PublicQRVerification.jsx`)

- **Purpose**: Public verification endpoint accessible by scanning a citizen's dynamic QR code or navigating to `/verify?token=...`.
- **Algorithm**:

1. Extract `token` parameter from URL search params.

2. Send `GET /api/card/verify-qr/:token`.

3. **Verification Logic**:

- If token starts with `CIV-TOKEN-` or matches active card, server returns status `🟢 Verified Identity`, citizen name, masked Aadhaar, and SHA-256 cryptographic signature.

- Otherwise, server returns status `🔴 Invalid / Expired Credential`.

4. Render official verification badge and timestamped verification report.

3.7 Organization Verification Portal (`OrganizationPortal.jsx`)

- **Purpose**: Enterprise interface for employers, banks, and institutions to verify citizen credentials with explicit consent.
- **Algorithm**:

1. Organization inputs Request Token or Scanned Dynamic QR Code.

2. Send verification request to server.

3. Server validates authorization consent and returns cryptographic verification badge with validity window.

3.8 Government Officer Portal (`AuthorityPortal.jsx`)

- **Purpose**: Official verification and document issuance workspace for government department officers.
- **Algorithm**:

1. Render officer header with department badge (`LEVEL-3 VERIFIED`).

2. **Search Citizen Identity**:

- Enter Civic ID or Aadhaar number to look up citizen records.

3. **Issue New Official Credential**:

- Form collects Citizen ID, Document Type, and Expiry Date.

- Submitting issues document directly into citizen's vault with official government seal.

3.9 National Super Admin Control Center (`AdminPortal.jsx`)

- **Purpose**: Master control dashboard for system administrators.
- **Algorithm**:

1. **System Health Metrics**: Display real-time CPU/Memory usage, active sessions, API response times, and error rates.

2. **Pending Gold Pass Approvals**:

- Fetch list via `GET /api/admin/goldpass/requests`.

- Admin can approve, reject, or revoke Gold Pass applications via `POST /api/admin/goldpass/verify`.

3. **Security Audit Logs**:

- Stream live system audit logs from `db.securityLogs` tracking logins, uploads, verifications, and payment webhooks.

3.10 Supporting Views (`VoterIdVault.jsx`, `SecurityCentre.jsx`, `ProfileSettings.jsx`, `HelpCentre.jsx`)

- **Voter ID Vault**: Manages election identity, polling station locator, and digital voter slip download.
- **Security Centre**: Shows security score (e.g. 98/100), connected devices, active sessions, and 2FA settings.
- **Profile Settings**: Enables updating citizen contact details, emergency contacts, and notification preferences.
- **Help Centre & AI Assistant (`AiAgentFloating.jsx`)**: Provides FAQ search and an interactive AI chatbot assisting citizens with portal navigation.

---

4. Summary Table of Login Routes & Gateways

Portal / Gateway	Access Route	Authentication Required	Verification Mechanism
Citizen Pre-Entry Gate	Main Button / `/gate`	Mobile OTP + Aadhaar OTP	2-Factor OTP + Tokenized UIDAI
Government Officer Gate	`#authority` / `/authority`	Dept Email + Officer Passcode	Level-3 Officer Clearance (`govt123`)
Super Admin Gate	`#admin` / `Ctrl+Shift+A`	Master Username + Passkey + Hardware Token	Master Root Clearance (`superadmin123`)
Public QR Verification	`/verify?token=...`	None (Public Endpoint)	SHA-256 Dynamic QR Token Verification

---